

How Remi's terms relate mathematically

A walkthrough with a three-axis toy example · S1 ONTOLOGY · 2026-09-16

0. The one idea everything rests on

Every sensory word is a coordinate axis. Remi has 216 lexicon words, so a product's sensory state is a point in a 216-dimensional space. To keep the example readable we use **three axes**:

```
L-space (toy)   axis 1 = L.ta.sweet      axis 2 = L.tx.thickness    axis 3 = L.ta.sour

a score vector  s = ( s_sweet , s_thick , s_sour )      each entry is a ±3 panel score
the origin      o = ( 0 , 0 , 0 )                      = the benchmark: "same as reference"
```

That is why the field is called `axis`: an `L` id names one coordinate.

1. Node → vector: what one ingredient "is" to the model

An ingredient node has an id and nothing else. What the model uses is its **vector**: one number per axis, "how far a typical dose moves the score". In YAML we store only the non-zero entries, each as a **component** record:

```
ING.xanthan_gum
effects:
- {axis: L.tx.thickness, direction: increase,
  magnitude: strong}      # +1.0
- {axis: L.ta.sweet,      direction: decrease,
  magnitude: weak}        # -0.3 (gum masks sweetness)
```

the same thing as a vector

```
v(xanthan) = ( -0.3 , +1.0 , 0 )
              sweet  thick  sour
```

words → numbers (loader):
weak 0.3 · medium 0.6 · strong 1.0
decrease → minus sign

`component` one entry, e.g. "+1.0 on thickness" — the record carries its axis, sign, size, scope, confidence. `vector` the whole list = the ingredient's row of numbers. A zero is simply an axis that is not listed.

2. Roles: where most of an ingredient's vector comes from

Ingredients rarely list all their effects themselves. They inherit the vector of each **role** (`FT`) they belong to, scaled by the role weight, and then override what is special to them:

$$v(ING) = \sum \text{over roles } w_{\text{role}} \cdot v(FT_{\text{role}}) + \text{overrides}$$

```
sugar : roles = [ FT.sweetener (strong) ]
v(FT.sweetener) = ( +1.0 , +0.3 , 0 )      sweeteners add sweetness and a little body
v(sugar)        = 1.0 · ( +1.0 , +0.3 , 0 ) = ( +1.0 , +0.3 , 0 )
```

```
xanthan : roles = [ FT.thickener (strong) ]
v(FT.thickener) = ( 0 , +1.0 , 0 )
override on sweet: -0.3
v(xanthan)      = ( -0.3 , +1.0 , 0 )
```

This is the INGREDIENT layer's 2-tier design: shared numbers live on the role, exceptions on the ingredient. Role weights stay words (weak/medium/strong); learned numbers land on the ingredient's own components (rule A10).

3. Vectors stacked = the matrix Γ

Put one ingredient per row and one axis per column. That table is Γ — the thing the model actually multiplies.

Γ	sweet	thick	sour
sugar	+1.0	+0.3	0
xanthan	-0.3	+1.0	0

matrix is never written in YAML. It is what you get by stacking every ingredient's vector; the loader builds it. Real size: ~118 ingredients $\times$ 216 axes.

4. The three multiplications

4a. Predict — stage D1: "if I change the recipe, how do the scores move?"

The recipe change is a vector $\mathbf{x}$ with one entry per ingredient, in units of "typical doses added". Add one dose of sugar and half a dose of xanthan:

```
x = ( +1.0 , +0.5 )          (sugar, xanthan)

Δs = xT Γ                    row-vector × matrix

Δs_sweet = 1.0·(+1.0) + 0.5·(-0.3) = +0.85
Δs_thick = 1.0·(+0.3) + 0.5·(+1.0) = +0.80
Δs_sour  = 1.0·( 0 ) + 0.5·( 0 ) = 0

Δs = ( +0.85 , +0.80 , 0 )    "a bit sweeter, noticeably thicker, sourness unchanged"
```

Each score is a sum down one column: every ingredient's contribution to that axis, weighted by how much of it you added. This is the whole of "build a theoretical recipe" — the optimiser just searches for the $\mathbf{x}$ whose Δs is closest to the target.

4b. Interact — R-3: "perceptions also shift each other"

Some axes change other axes on their own — sweetness suppresses sourness. Those are the **IN** scalars, entries of an axis-by-axis matrix M . Our toy has one:

M	→ sweet	→ thick	→ sour
sweet →	0	0	-0.6
thick →	0	0	0
sour →	0	0	0

```
s' = s + M s

sour' = 0 + (-0.6)·(+0.85) = -0.51

s' = ( +0.85 , +0.80 , -0.51 )
"...and it will taste a little less sour"
```

scalar one **IN** record = one cell of M . The matrix is almost empty on purpose (rule C4: only well-established causal interactions).

4c. Measure — stage E1: "what would the lab see, and does it agree?"

A parameter is a lab reading that stands in for some axes. Its `axes` list is a vector over L ; stacked, they form Π . Viscosity stands in for thickness:

Π	sweet	thick	sour
P.apparent_viscosity	0	+1.0	0
P.pH	0	0	-1.0

$$m = \Pi s$$

`m_viscosity` = +0.80 (expected reading, scaled)

`m_pH` = +0.51 (less sour → higher pH)

if the lab reads viscosity $\approx$ +0.8:

confidence in `s_thick` goes up (rule A4)

if it reads something else:

the `xanthan→thick` component is corrected (E1)

The model runs on Δs alone; Π is only used when a reading exists. That is what "P is auxiliary" means in equations.

5. The tuple switches components on and off

The same ingredient does not behave the same in every product. Each component carries a **scope**; the profile tuple decides which components are active, so Γ is really $\Gamma(t)$:

```
xanthan → thickness : {scope: SC.emulsion.ow|APP.sauce,    magnitude: strong} → +1.0
                      {scope: SC.emulsion.ow|APP.beverage,  magnitude: medium} → +0.6
```

```
t = [SC.emulsion.ow, APP.sauce,    ST.ambient] →  $\Gamma_t$ [xanthan, thick] = +1.0
```

```
t = [SC.emulsion.ow, APP.beverage, ST.ambient] →  $\Gamma_t$ [xanthan, thick] = +0.6
```

matching rule (B5): a scope is "on" when every part it writes equals the profile's part;
a blank part matches anything. If two scopes match, the one with more parts written wins (B6).

tuple the profile `[SC, APP, ST]` is not a number — it is a selector. It never enters the multiplication; it chooses which numbers do.

6. Where the product sits

```
product = ( t , key ingredients , identity axes  $I \subset L$  , origin o )
```

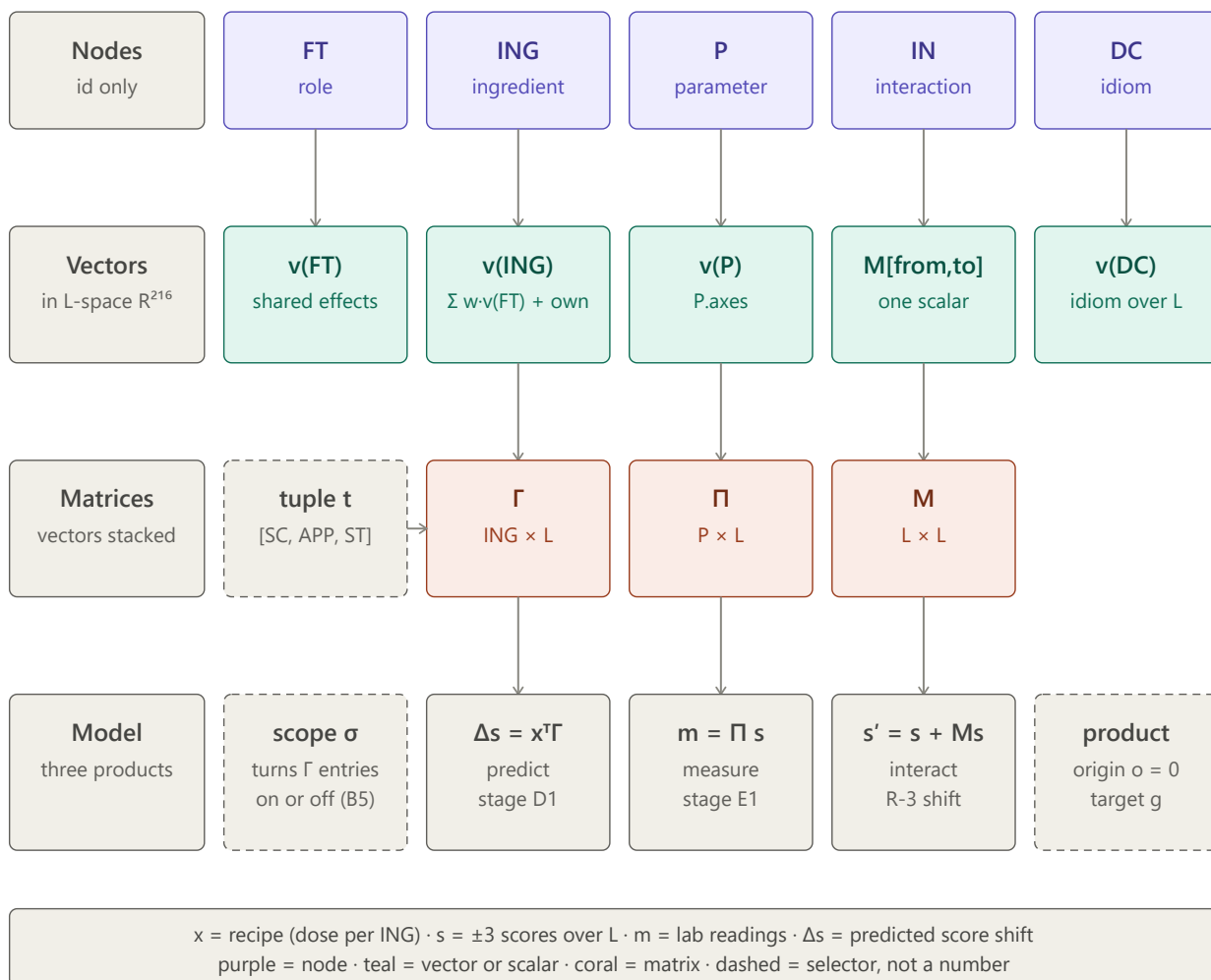
`o` = the reference sample; all scores are 0 there

`g` = the goal vector the user asked for, e.g. `g = ( -1 , +1 , 0 )` "less sweet, thicker"

D1 searches for `x` such that $\Delta s(x) \approx g$ subject to ranges and the palette

Two products with the same tuple but different key ingredients have different Γ rows in play and different origins — that is the "same ruler" rule (B9): scores are only comparable when `t`, key ingredients and `o` all match.

7. The map



Read one column at a time: **ING** owns **v(ING)**; those vectors stacked are **Γ**; **Γ** is what the model multiplies by the recipe. **FT** and **DC** stop at the vector row — a role's vector is only inherited, an idiom's vector is a target direction. Dashed boxes are selectors, not numbers.

8. Glossary, tied to the example

Word	In the example	In the files
node	"sugar", "thickness", "viscosity" — names	an id line: ING.sucrose , L.tx.thickness , P.apparent_viscosity
component	+1.0 on thickness	one {axis, direction, magnitude, ...} record
vector	v(xanthan) = (-0.3, +1.0, 0)	the effects list of one ingredient (or axes of one P, one DC)
scalar	M[sweet, sour] = -0.6	one IN record
matrix	Γ, M, Π tables above	not stored; built by the loader from all vectors
tuple	[SC.emulsion.ow, APP.sauce, ST.ambient]	a profile entry; a scope string on a component
sentence	"gum masks sweetness — measured 2026-08"	a LESSON (observation) or RP (prescription)

Remi 용어의 수학적 관계

세 축 예제로 따라가는 설명 · S1 ONTOLOGY · 2026-09-16

0. 모든 것이 서 있는 한 가지 생각

감각 낱말 하나가 좌표축 하나다. Remi 의 렉시콘 낱말은 216개이므로, 한 제품의 감각 상태는 216차원 공간의 한 점이다. 읽기 쉽게 예제는 **축 세 개**만 쓴다:

L-공간 (예제) 축 1 = L.ta.sweet (단맛) 축 2 = L.tx.thickness (겉쪽함) 축 3 = L.ta.sour (신맛)

점수 벡터 $s = (s_sweet, s_thick, s_sour)$ 각 성분은 패널의 ± 3 점수
원점 $o = (0, 0, 0)$ = 기준 제품: "기준과 같다"

필드 이름이 `axis` 인 이유가 이것이다: `L` id 하나가 좌표축 하나를 가리킨다.

1. 노드 → 벡터: 모형에게 재료 하나는 무엇인가

재료 노드는 id 만 가진다. 모형이 쓰는 것은 그 재료의 **벡터**다: 축마다 숫자 하나, "통상량을 넣으면 점수가 얼마나 움직이나". YAML 에는 0 이 아닌 성분만 **성분(component)** 레코드로 적는다:

```
ING.xanthan_gum
effects:
- {axis: L.tx.thickness, direction: increase,
  magnitude: strong}          # +1.0
- {axis: L.ta.sweet,      direction: decrease,
  magnitude: weak}           # -0.3 (검이 단맛을 가린다)
```

같은 것을 벡터로 쓰면

$v(\text{잔탄}) = (-0.3, +1.0, 0)$
 단맛 겉쪽함 신맛

낱말 → 숫자 (로더):
weak 0.3 · medium 0.6 · strong 1.0
decrease → 음의 부호

성분 항목 하나, 예: "겉쪽함에 +1.0" — 레코드는 축·부호·크기·스코프·확신도를 함께 든다. **벡터** 목록 전체 = 그 재료의 숫자 한 줄. 0 은 적히지 않은 축일 뿐이다.

2. 역할: 재료 벡터의 대부분이 오는 곳

재료가 효과를 전부 스스로 적는 일은 드물다. 속한 **역할**(`FT`)의 벡터를 역할 무게만큼 물려받고, 자기만 다른 것을 덮어쓴다:

$$v(ING) = \sum \text{역할마다 } w_{\text{역할}} \cdot v(FT_{\text{역할}}) + \text{overrides}$$

설탕 : 역할 = [FT.sweetener (strong)]
 $v(FT.sweetener) = (+1.0, +0.3, 0)$ 감미료는 단맛과 약간의 바디를 더한다
 $v(\text{설탕}) = 1.0 \cdot (+1.0, +0.3, 0) = (+1.0, +0.3, 0)$

잔탄 : 역할 = [FT.thickener (strong)]
 $v(FT.thickener) = (0, +1.0, 0)$
 단맛 override: -0.3
 $v(\text{잔탄}) = (-0.3, +1.0, 0)$

INGREDIENT 층의 2단 설계가 이것이다: 공통 숫자는 역할에, 예외는 재료에. 역할 무게는 낱말(weak/medium/strong)로 두고, 학습된 숫자는 재료 자신의 성분에 쌓인다(규칙 A10).

3. 벡터를 쌓으면 = 행렬 Γ

한 행에 재료 하나, 한 열에 축 하나. 그 표가 Γ — 모형이 실제로 곱하는 것이다.

Γ	단맛	겉쪽함	신맛
설탕	+1.0	+0.3	0
잔탄	-0.3	+1.0	0

행렬 은 YAML 에 적히지 않는다. 모든 재료의 벡터를 쌓으면 나오는 것이고, 로더가 만든다. 실제 크기: 재료 약 118 × 축 216.

4. 곱셈 셋

4a. 예측 — D1 단계: "배합을 바꾸면 점수가 어떻게 움직이나?"

배합 변화는 재료마다 성분 하나인 벡터 x 이고 단위는 "통상량 몇 배를 더 넣었나". 설탕 한 배, 잔탄 반 배를 더 넣으면:

$$x = (+1.0, +0.5) \quad (\text{설탕, 잔탄})$$

$$\Delta s = x^T \Gamma \quad \text{행벡터 } x \text{ 행렬 } \Gamma$$

$$\Delta s_{\text{단맛}} = 1.0 \cdot (+1.0) + 0.5 \cdot (-0.3) = +0.85$$

$$\Delta s_{\text{겉쪽함}} = 1.0 \cdot (+0.3) + 0.5 \cdot (+1.0) = +0.80$$

$$\Delta s_{\text{신맛}} = 1.0 \cdot (0) + 0.5 \cdot (0) = 0$$

$$\Delta s = (+0.85, +0.80, 0) \quad \text{"조금 더 달고, 뚜렷이 더 겉쪽하고, 신맛은 그대로"}$$

점수 하나는 열 하나를 세로로 더한 것이다: 그 축에 대한 모든 재료의 기여를 넣은 양만큼 가중해 더한다. "이론 배합을 만든다" 는 것이 이것의 전부다 — 최적화기는 Δs 가 목표에 가장 가까운 x 를 찾을 뿐이다.

4b. 상호작용 — R-3: "느낌은 서로 움직인다"

어떤 축은 다른 축을 스스로 바꾼다 — 단맛은 신맛을 누른다. 그것이 $\mathbf{IN}$ 스칼라이고, 축×축 행렬 M 의 칸이다. 예제에는 하나만 있다:

M	→ 단맛	→ 걸쭉함	→ 신맛
단맛 →	0	0	-0.6
걸쭉함 →	0	0	0
신맛 →	0	0	0

$$s' = s + M s$$

$$\text{신맛}' = 0 + (-0.6) \cdot (+0.85) = -0.51$$

$$s' = (+0.85, +0.80, -0.51)$$

"...그리고 조금 덜 시게 느껴진다"

스칼라 IN 레코드 하나 = M 의 칸 하나. 이 행렬은 일부러 거의 비어 있다(규칙 C4: 확립된 인과 상호작용만).

4c. 측정 — E1 단계: "실험실은 무엇을 보고, 그것이 맞나?"

측정 지표는 어떤 측들을 대신하는 실험실 측정값이다. 그 axes 목록이 L 위의 벡터이고, 쌓으면 π 가 된다. 점도는 걸쭉함을 대신한다:

π	단맛	걸쭉함	신맛
P.apparent_viscosity	0	+1.0	0
P.pH	0	0	-1.0

$$m = \pi s$$

$$m_{\text{점도}} = +0.80 \quad (\text{기대 측정값, 척도 맞춤})$$

$$m_{\text{pH}} = +0.51 \quad (\text{덜 시다} \rightarrow \text{pH 높다})$$

실험실 점도가 $\approx +0.8$ 이면:

$s_{\text{걸쭉함}}$ 의 확신도가 오른다 (규칙 A4)

다른 값이 나오면:

잔탄→걸쭉함 성분을 고친다 (E1)

모형은 Δs 만으로 돈다; π 는 측정값이 있을 때만 쓴다. "P 는 보조" 라는 말을 식으로 쓰면 이것이다.

5. 튜플이 성분을 켜고 끈다

같은 재료가 모든 제품에서 같게 행동하지는 않는다. 성분마다 스코프가 있고, 제형 튜플이 어느 성분이 켜지는지 정한다. 그래서 Γ 는 사실 $\Gamma(t)$ 다:

잔탄 → 걸쭉함 : {scope: SC.emulsion.ow|APP.sauce, magnitude: strong} → +1.0
 {scope: SC.emulsion.ow|APP.beverage, magnitude: medium} → +0.6

$t = [\text{SC.emulsion.ow}, \text{APP.sauce}, \text{ST.ambient}] \rightarrow \Gamma_t[\text{잔탄, 걸쭉함}] = +1.0$

$t = [\text{SC.emulsion.ow}, \text{APP.beverage}, \text{ST.ambient}] \rightarrow \Gamma_t[\text{잔탄, 걸쭉함}] = +0.6$

맞춤 규칙 (B5): 스코프에 적힌 칸마다 제형의 그 칸과 같으면 "켜짐";

비운 칸은 무엇이든 맞는다. 둘이 맞으면 더 많은 칸을 적은 쪽이 이긴다 (B6).

튜플 제형 [SC, APP, ST] 은 숫자가 아니라 선택자다. 곱셈에 들어가지 않고, 어느 숫자가 들어갈지를 고른다.

6. 제품은 어디 있나

제품 = (t , 핵심 재료 , 정체성 축 $I \subset L$, 원점 o)

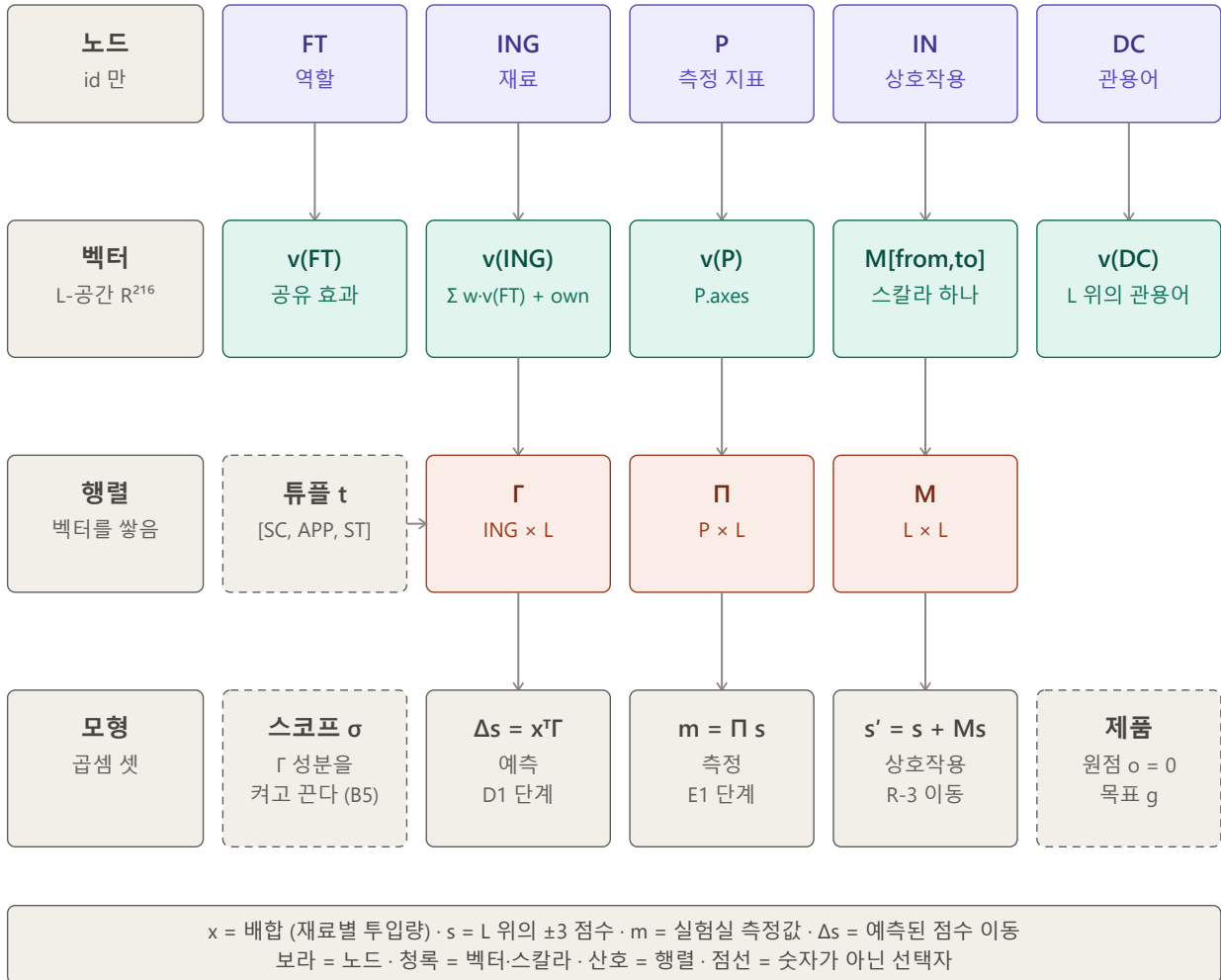
o = 기준 시료; 거기서 모든 점수는 0

g = 사용자가 원하는 목표 벡터, 예: $g = (-1, +1, 0)$ "덜 달게, 더 걸쭉하게"

D1 은 범위와 팔레트 안에서 $\Delta s(x) \approx g$ 인 x 를 찾는다

튜플은 같고 핵심 재료가 다른 두 제품은 쓰이는 Γ 행이 다르고 원점도 다르다 — "같은 자" 규칙(B9)이 이것이다: $t \cdot$ 핵심 재료 $\cdot o$ 가 모두 같을 때만 점수를 비교할 수 있다.

7. 지도



한 번에 한 열씩 읽는다: **ING** 는 **$v(ING)$** 를 가지고, 그 벡터들을 쌓으면 Γ , Γ 는 모형이 배합과 곱하는 것이다. **FT** 와 **DC** 는 벡터 줄에서 멈춘다 — 역할의 벡터는 물려줄 뿐이고, 관용어의 벡터는 목표 방향이다. 점선 상자는 숫자가 아니라 선택자다.

8. 용어 — 예제와 붙여서

날말	예제에서	파일에서
노드 (node)	"설탕", "겔쥙함", "점도" — 이름	id 한 줄: <code>ING.sucrose</code> , <code>L.tx.thickness</code> , <code>P.apparent_viscosity</code>
성분 (component)	겔쥙함에 +1.0	<code>{axis, direction, magnitude, ...}</code> 레코드 하나
벡터 (vector)	$v(\text{잔탄}) = (-0.3, +1.0, 0)$	재료 하나의 <code>effects</code> 목록 (또는 P 하나의 <code>axes</code> , DC 하나)
스칼라 (scalar)	$M[\text{단맛, 신맛}] = -0.6$	<code>IN</code> 레코드 하나
행렬 (matrix)	위의 Γ , M , Π 표	저장하지 않는다; 로더가 모든 벡터에서 만든다
튜플 (tuple)	<code>[SC.emulsion.ow, APP.sauce, ST.ambient]</code>	제형 항목 하나; 성분에 붙는 스코프 문자열
문장 (sentence)	"검이 단맛을 가린다 — 2026-08 관찰"	<code>LESSON</code> (관찰) 또는 <code>RP</code> (처방)